武汉大学 国家网络安全学院
SCHOOL OF CYBER SCIENCE AND ENGINEERING·WHU

信息系统安全

（Information System Security）

武汉大学
王鹃

jwang@whu.edu.cn

目录

- 信息系统安全概述
- 软件安全与恶意代码
- 渗透测试方法及工具
- 模糊测试与漏洞挖掘
- 用户身份认证机制
- 访问控制模型
- 操作系统基础安全机制
- 硬件安全与可信计算
- 系统风险评估方法与工具
- 信息系统安全研究新方向

第四章 模糊测试与漏洞挖掘

LISS
空天信息安全与可信计算
教育部重点实验室

目录

1 模糊测试简介

2 模糊测试方法

3 模糊测试器类型

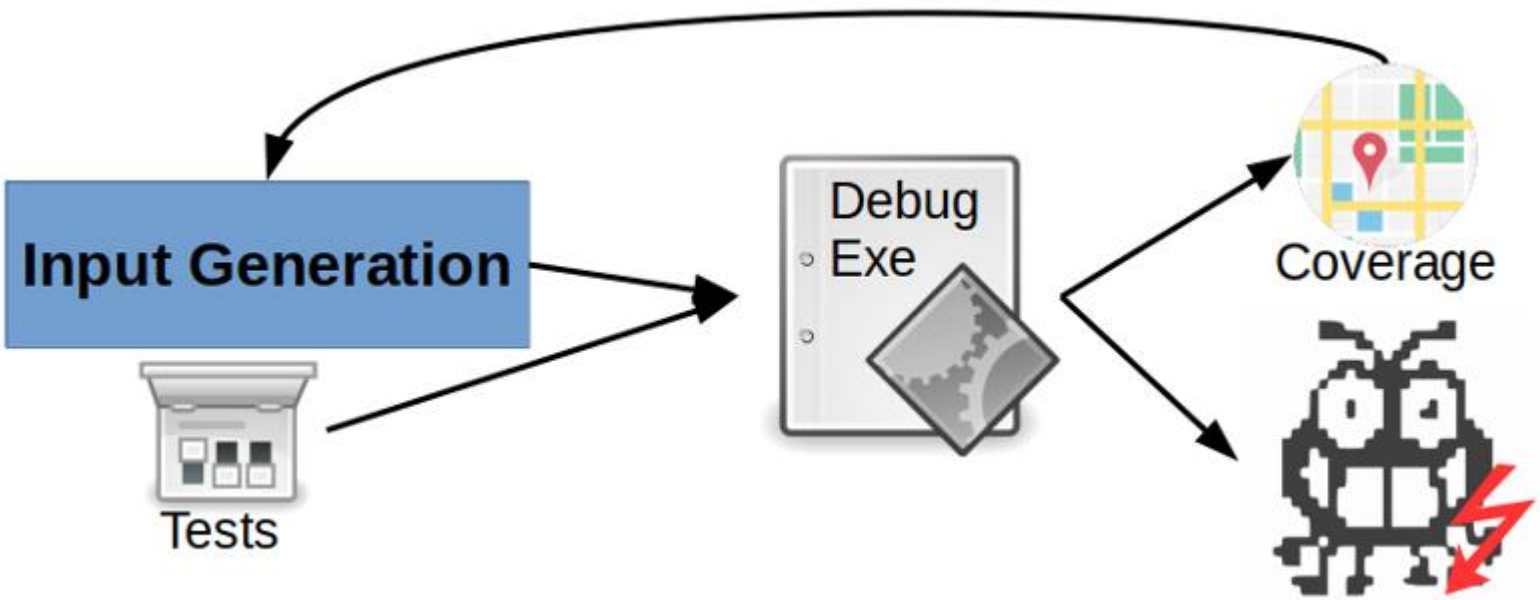
4 程序插桩原理及实例

5 模糊测试工具及实例

6 模糊测试的局限性

1、模糊测试简介

- 模糊测试（Fuzz testing）是一种发现安全漏洞的有效的测试方法，通过向目标系统提供非预期的输入并监视异常结果来发现软件漏洞。



1、模糊测试简介

- 模糊测试将随机的坏数据插入程序，观察程序是否能容忍杂乱输入。

- 模糊测试产生杂乱数据攻击程序。采用模糊测试攻击应用程序可发现其他采用逻辑思维来测试很难发现的安全漏洞。

1、模糊测试简介

- 模糊测试
  - 1988年由威斯康辛大学Bart Miller 提出
  - 假设: 程序可以处理任何输入
  - 结果: 发现 crash 25-33% of UNIX utility programs

1、模糊测试简介

例子：

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char buffer[10] = "Hello!";
    char string[100];
    scanf("%s", string);
    strcpy(buffer, string);

    printf("%s %s\n", buffer, string);
}
```

1、模糊测试简介

```
                    american fuzzy lop 2.56b (test.o)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 4 min, 14 sec       │  cycles done : 303    │
│   last new path : none yet (odd, check syntax!)      │  total paths : 1      │
│ last uniq crash : 0 days, 0 hrs, 2 min, 21 sec       │ uniq crashes : 2      │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 0 (0.00%)         │    map density : 0.00% / 0.00%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 1 (100.00%)            │
│ stage execs : 90/256 (35.16%)       │  new edges on : 1 (100.00%)            │
│ total execs : 78.5k                 │ total crashes : 5936 (2 unique)        │
│  exec speed : 312.5/sec             │  total tmouts : 5 (1 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 0/8, 0/7, 0/5                         │    levels : 1          │
│  byte flips : 0/1, 0/0, 0/0                         │   pending : 0          │
│ arithmetics : 0/56, 0/0, 0/0                        │  pend fav : 0          │
│  known ints : 0/4, 0/0, 0/0                         │ own finds : 0          │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 2/78.3k, 0/0                          │ stability : 100.00%    │
│        trim : n/a, 0.00%                            ├────────────────────────┘
└^C[!] WARNING: error waitpid                         │          [cpu: 36%]
```

1、模糊测试简介

id:000000,sig:04,src:000000,op:havoc,rep:4 × id:000001,sig:06,src:000000,op:havoc,rep:128 ×

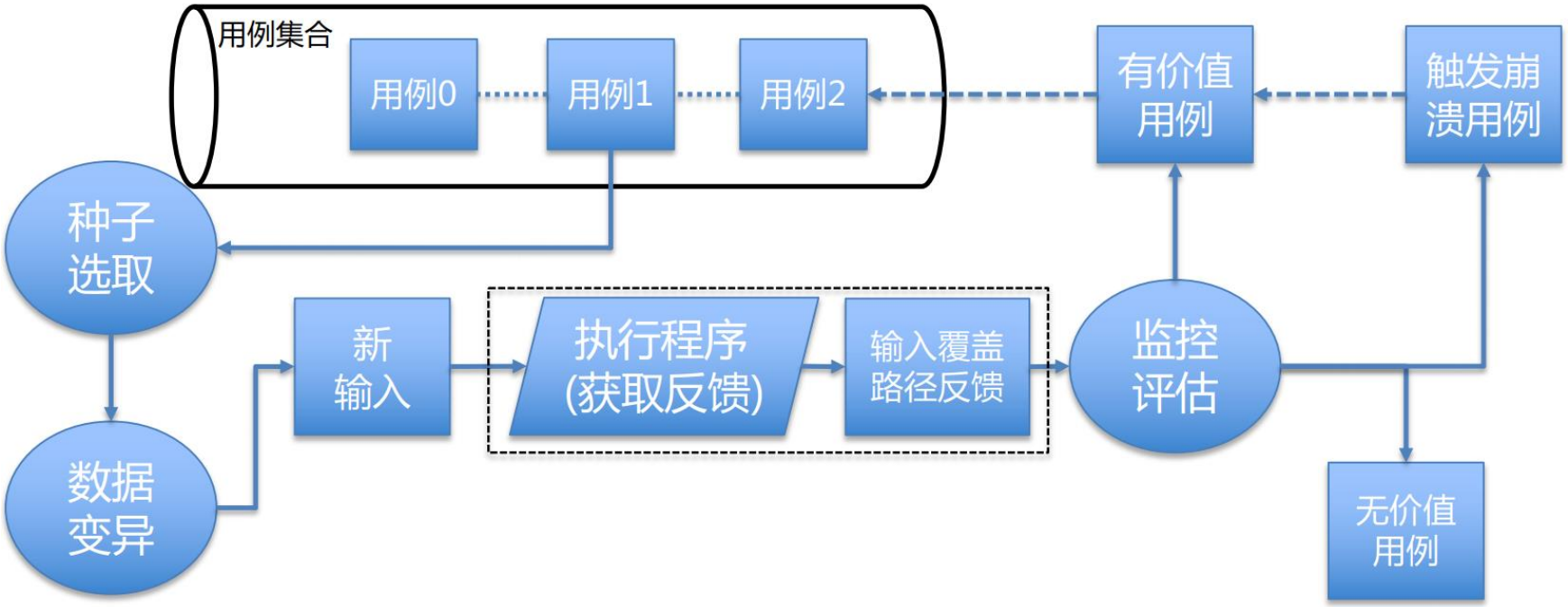
```
<0x05><0x05><0x05><0x05><0x05><0x05><0x05><0x05><0x05><0x05>
<0x05><0x05><0x05><0x05><0x05>a
```

id:000000,sig:04,src:000000,op:havoc,rep:4 × id:000001,sig:06,src:000000,op:havoc,rep:128 ×

```
b1de d6de dede ff00 f8de ff00 0000 8000
7fd6 ded6 40de dede de5f 7200 7fe9 ded6
de7b 001c 8000 7fd6 ded6 dede dad5 de5f
8000 7fe9 64d6 de7f d8ff ff7f ffff dede
ff00 8000 7fd6 ded6 def8 deff 00de d6de
7fde dede 5f6d 007f dede 5f80 00e9 ded6
de7f d8ff ff7f ff7f d6de d640 dede dede
5f80 00ff dede ff00 8000 7fd6 ded6 def8
deff 00de d680 007f dc
```

2、模糊测试方法

模糊测试主要过程：



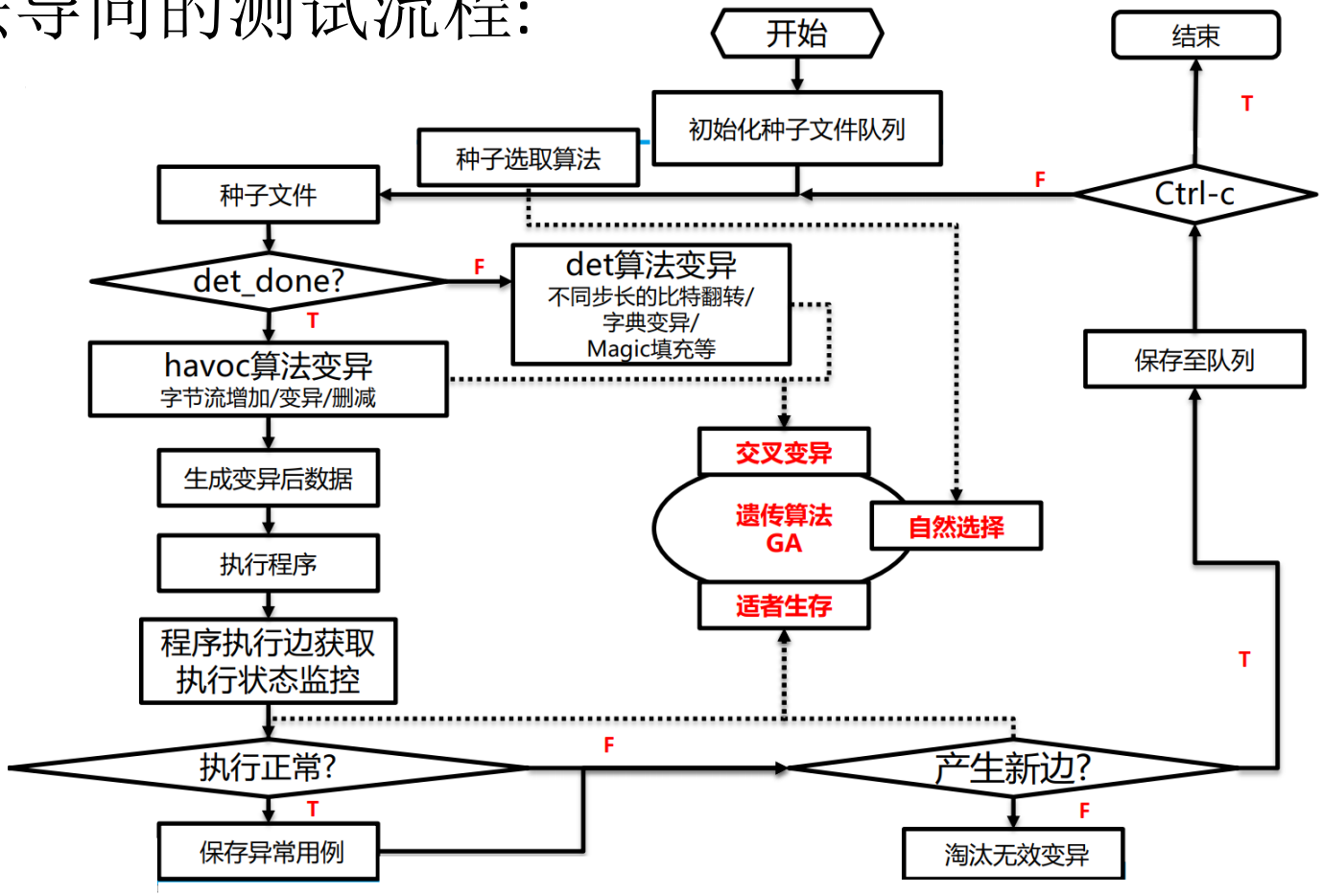
2、模糊测试方法

Fuzz测试中的主要方法

术语	定义
傻瓜(Dumb)Fuzz	随机破坏数据包，不考虑数据结构
智能(Smart)Fuzz	破坏数据包，但考虑数据结构，以及编码方法，如BASE-64编码，以及数据块间的关系，如校验值，某些数据域是否存在的标志位，数据位的长度
黑盒Fuzz	发送畸形的数据，不验证代码路径是否满足
白盒Fuzz	发送畸形的数据，验证目标代码路径是否满足
生成(Generation)	自动产生Fuzz测试数据，并不基于任何前面的输入
变异(Mutation)	根据缺陷模式，通过破坏有效的数据来产生Fuzz测试数据
变异模板	使用变异模板作为输入，产生Fuzz缓冲池并发送给测试软件

2、模糊测试方法

基于遗传算法导向的测试流程:



LISS
空天信息安全与可信计算
教育部重点实验室

遗传算法

遗传算法模拟自然选择和自然遗传过程中发生的繁殖、交叉和基因突变现象，在每次迭代中都保留一组候选解，并按某种指标从候选解群中选取较优的个体，利用遗传算子(选择、交叉和变异)对这些个体进行组合，产生新一代的候选解群，重复此过程，直到满足某种收敛指标为止。

遗传算法

■ 遗传操作

亦称遗传算子(genetic operator)，就是关于染色体的运算。遗传算法中有三种遗传操作:

- 选择-复制(selection-reproduction)
- 交叉(crossover，亦称交换、交配或杂交)
- 变异(mutation，亦称突变)

遗传算法

■ **选择-复制** 通常做法是：对于一个规模为N的种群S，按每个染色体$x_i \in S$的选择概率$P(x_i)$所决定的选中机会，分N次从S中随机选定N个染色体，并进行复制。

这里的选择概率$P(x_i)$的计算公式为

$$P(x_i) = \frac{f(x_i)}{\sum_{j=1}^{N} f(x_j)}$$

LISS
空天信息安全与可信计算
教育部重点实验室

遗传算法

■**交叉** 就是互换两个染色体某些位上的基因。

例如，设染色体 s_1=01001011，s_2=10010101，交换其后4位基因，即

0100| 1011， 1001 | 0101

0100| 0101， 1001 | 1011

s_1'=01000101， s_2'=10011011

可以看做是原染色体s_1和s_2的子代染色体。

LISS
空天信息安全与可信计算
教育部重点实验室

遗传算法

变异 就是改变染色体某个(些)位上的基因。

例如，设染色体 s=11001101

将其第三位上的0变为1, 即

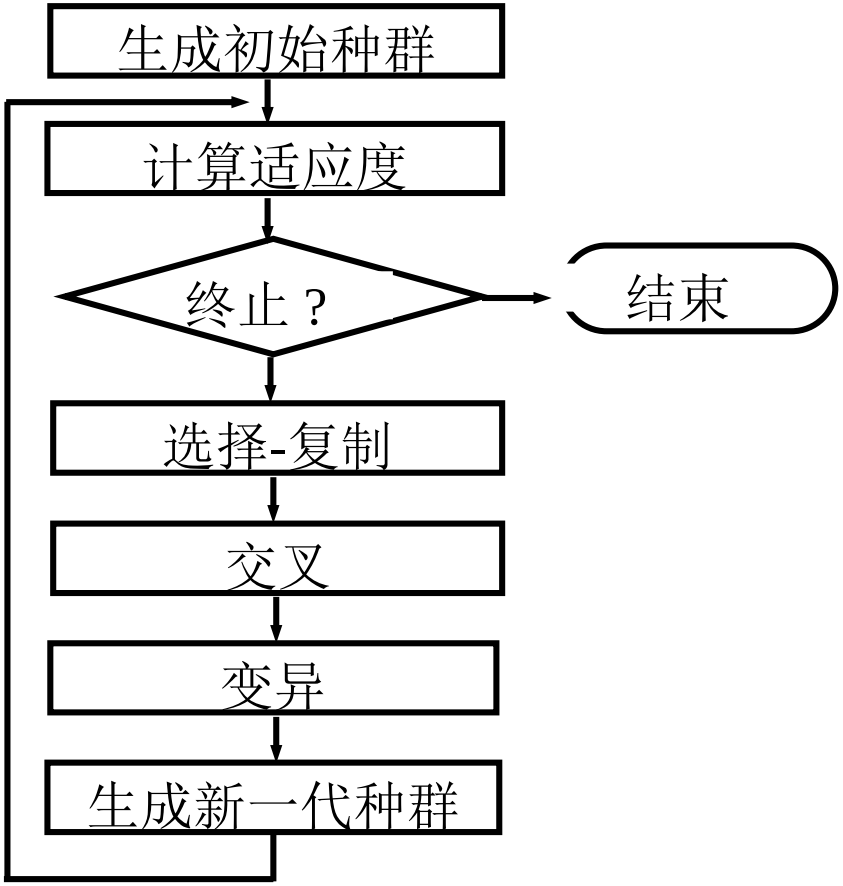
s=11<u>0</u>01101 →11<u>1</u>01101= s'。

s'也可以看做是原染色体s的子代染色体。

LISS
空天信息安全与可信计算
教育部重点实验室

遗传算法

基本遗传算法

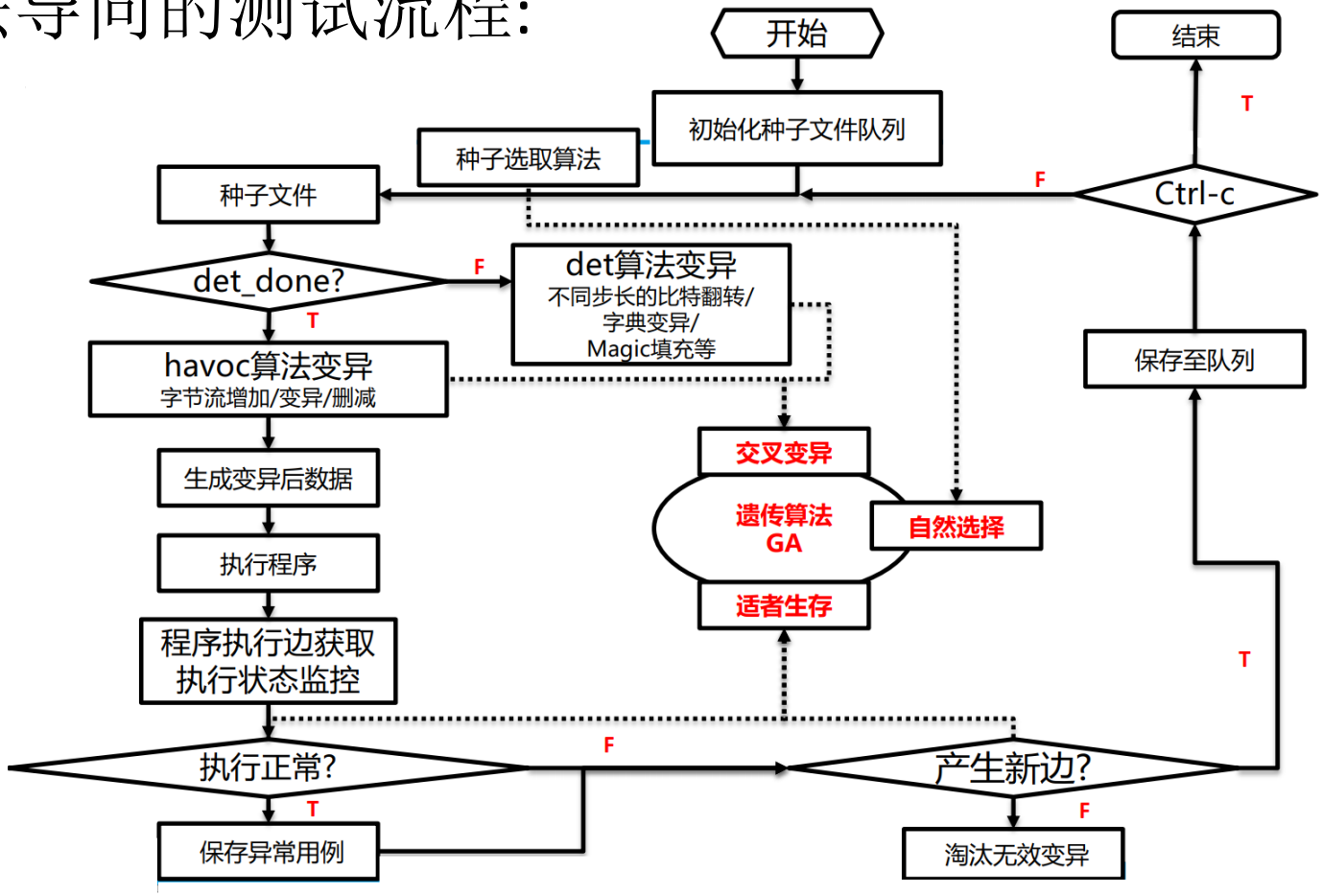


遗传算法基本流程框图

LISS
空天信息安全与可信计算
教育部重点实验室

2、模糊测试方法

基于遗传算法导向的测试流程:



LISS
空天信息安全与可信计算
教育部重点实验室

3、模糊测试器类型

模糊测试器类型：

- 文件Fuzz测试
- 协议Fuzz测试
- 组件Fuzz测试
- Web Fuzz测试
- 内存模糊测试

3、模糊测试器类型

文件Fuzz测试

- 文件作为程序的主要输入
- 按照事先约定好的数据结构对文件中不同的数据域进行解析
- 文件格式发生改变，攻击者进行修改并注入不安全代码。
- 发现程序崩溃、缓冲区溢出

例如：FileFuzz

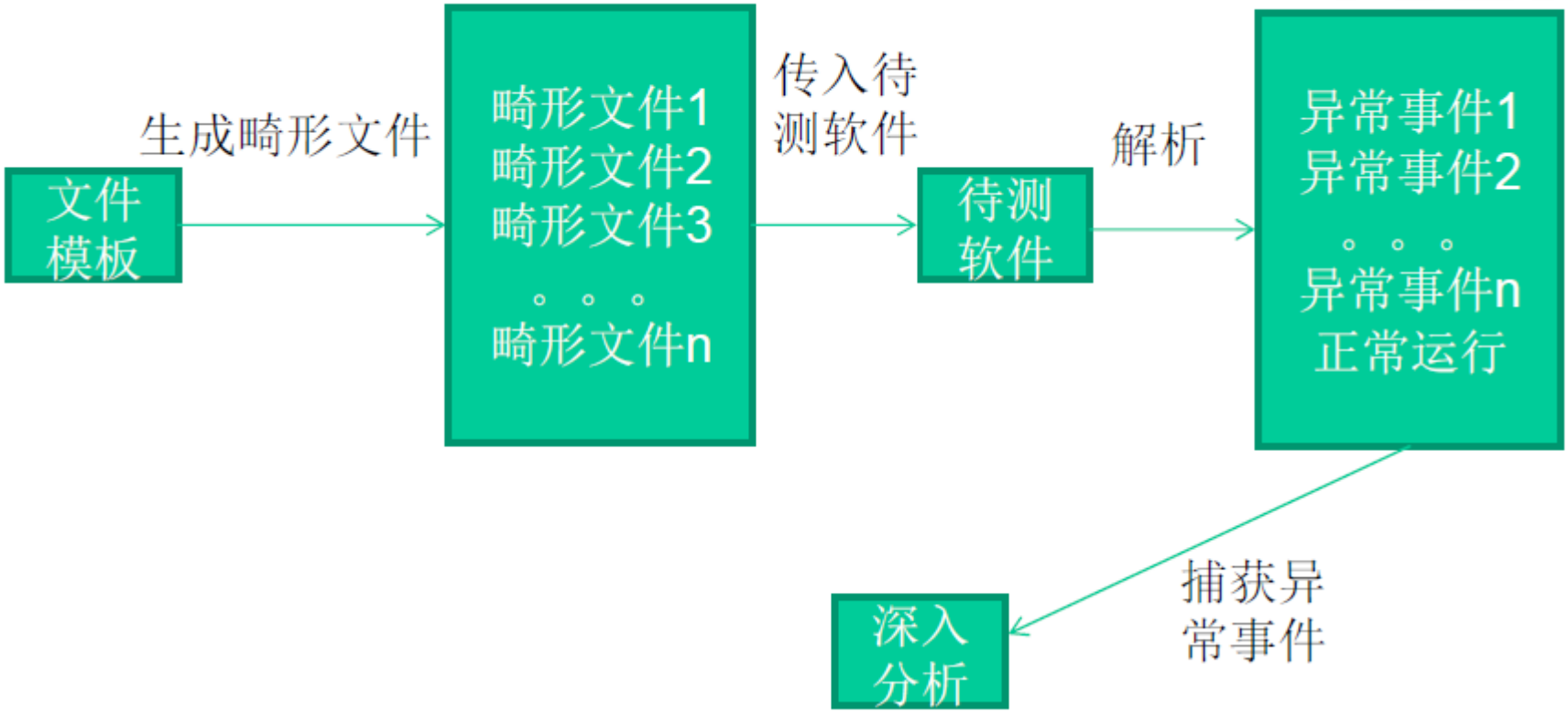
LISS
空天信息安全与可信计算
教育部重点实验室

3、模糊测试器类型

一般步骤

- 以一个正常的文件模板作为基础，按照一定规则产生一批畸形文件
- 将畸形文件逐一送入软件进行解析，并监视软件是否会抛出异常
- 记录软件产生的错误信息，如寄存器状态、栈状态等。
- 用日志或其他UI形式向测试人员展示异常信息，以进一步鉴定这些错误是否能被利用

3、模糊测试器类型


文件
模板
生成畸形文件
畸形文件1
畸形文件2
畸形文件3
。。。
畸形文件n
传入待
测软件
待测
软件
解析
异常事件1
异常事件2
。。。
异常事件n
正常运行
捕获异
常事件
深入
分析

3、模糊测试器类型

文件Fuzz测试漏洞挖掘类型：

- Blind Fuzz（盲模糊）
- Smart Fuzz（智能模糊）

3、模糊测试器类型

Blind Fuzz

- 在随机位置插入随机的数据以生成畸形文件
- 而现代软件往往使用非常复杂的私有数据结构，数据结构越复杂，解析程序的逻辑就越复杂，越容易出错
- 对于采用复杂数据结构的复杂文件进行漏洞挖掘，该方法暴露出一些不足
  - 产生测试用例的策略缺少针对性
  - 大量测试用例无效
  - 难以发现复杂解析器深层逻辑的漏洞

3、模糊测试器类型

复杂数据结构特征

- 拥有一批预定义的静态数据
- 数据结构的内容是可以动态改变的
- 数据结构之间是嵌套的
- 数据中存在多种数据关系(size of，point to，reference of，CRC)
- 有意义的数据被编码或压缩，甚至用另一种文件格式来存储，这些格式的文件被挖掘出来越来越多的漏洞

3、模糊测试器类型

Smart Fuzz

- 面向逻辑(Logic Oriented Fuzzing)
- 面向数据类型(Data Type Oriented Fuzzing)
- 基于样本(Sample Based Fuzzing)

3、模糊测试器类型

面向逻辑

- 测试前首先明确要测试的目标是解析文件的程序逻辑，而不是文件本身
- 复杂的文件格式往往要经过多“层”解析，因此还需要明确测试用例正在试探的是哪一层的解析逻辑，即明确测试“深度”以及畸形数据的测试“粒度”
- 明确了测试的逻辑目标后，在生成畸形数据时可以具有针对性的仅仅改动样本文件的特定位置，尽量不破坏其他数据依赖关系，这样使得改动的数据能够传递到测试的解析深度，而不会在上层的解析器中被破坏

3、模糊测试器类型

面向数据类型

- 算术型：包括以HEX、ASCII、Unicode、Raw格式存在的各种数值
- 指针型：包括Null指针、合法/非法的内存指针等
- 字符串型：包括超长字符串、缺少终止符(0x00)的字符串等
- 特殊字符：包括#，@，’，<,>,/,\,../等等

✓ 面向数据类型测试是指能够识别不同的数据类型，并且能够针对目标数据的类型按照不同规则来生成畸形数据，能够大大减少无效的畸形文件

3、模糊测试器类型

基于样本

- 测试前首先构造一个合法的样本文件（也叫模版文件），这时样本文件里所有数据结构和逻辑必然都是合法的。然后以这个文件为模板，每次只改动一小部分数据和逻辑来生成畸形文件，这种方法也叫做“变异”(Mutation)
- 对于复杂文件来说，以现成的样本文件为基础进行畸形数据变异来生成畸形文件的方法要比上面两种的难度要小很多，也更容易实现

但是这种方法不能测试样本文件里没有包含的数据结构

3、模糊测试器类型

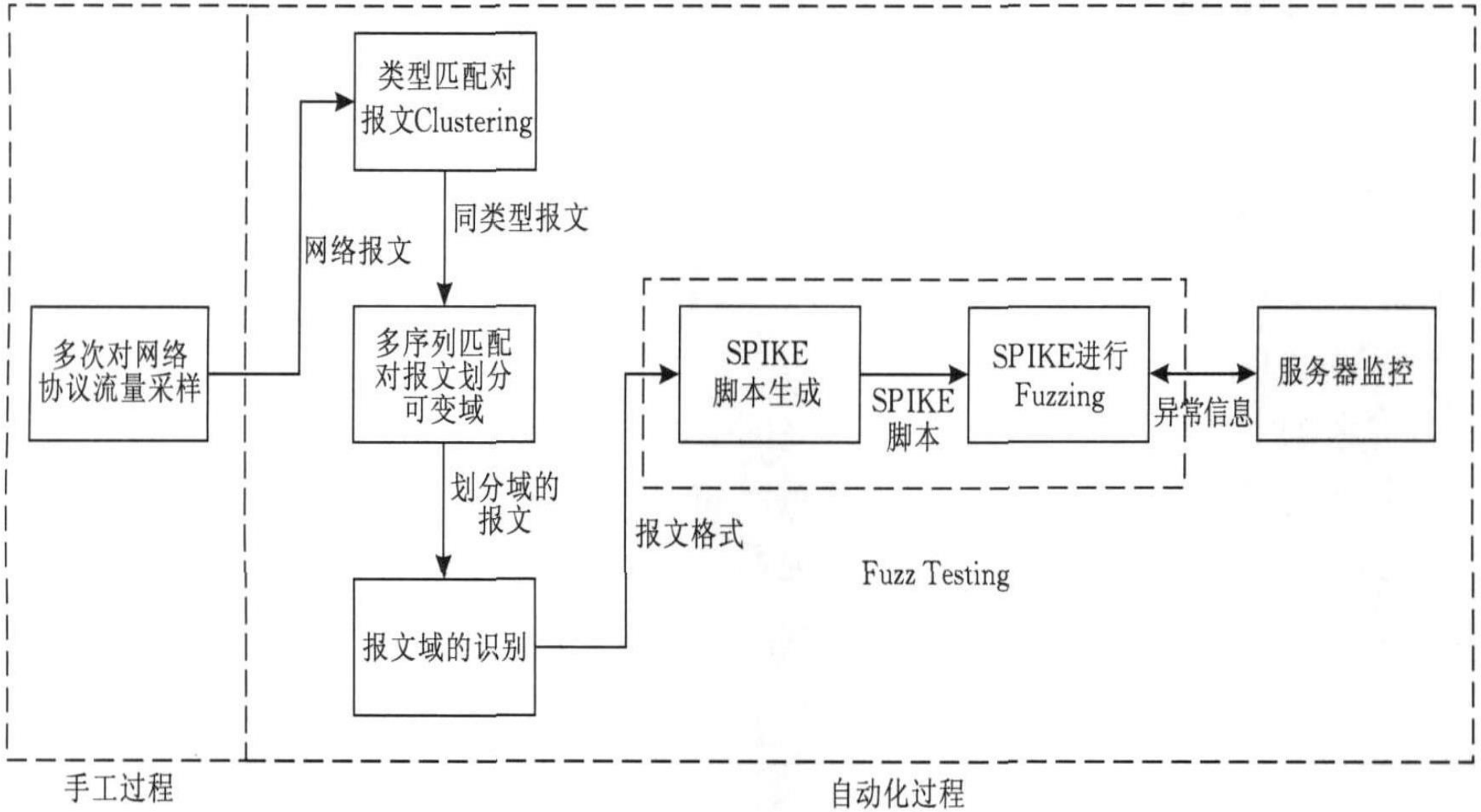
协议Fuzz测试

- 多数网络协议细节未公开，如SMB、TNS
- 存在拒绝服务、缓冲区溢出、格式化字符串
- 协议解析自动进行
- 协议解析的软件通常较复杂，需要反复进行数据的读取和处理

协议逆向工程：
Network Trace，利用网络流量来推测网络协议
Tainted Data，对实现网络协议的服务器程序进行二进制的动态跟踪分析，通过跟踪二进制文件对报文的处理流程对报文进行解析

3、模糊测试器类型

SPIKE



3、模糊测试器类型

组件Fuzz测试

- 组件提取
- 组件接口分析
- 构造测试用例
- 测试驱动、异常跟踪
- 结果分析

3、模糊测试器类型

Web Fuzz测试

- 特殊形式的网络协议模糊测试
- 测试方法
  - 设置目标环境
  - 输入
  - GET、POST、HEAD 、PUT 、DELETE 、TRACE 、CONNECT 、 OPTIONS
  - 请求URL
  - 异常检测

  如：Sqlmap

LISS
空天信息安全与可信计算
教育部重点实验室

3、模糊测试器类型

主要Web漏洞

- 拒绝服务（Denial-of-service,DoS）
- 跨站脚本（Cross-site scripting,XSS）
- SQL注入（SQL injection）
- 目录遍历/弱访问控制（Directory traversal/Weak access control）
- 弱认证（Weak authentication）
- 弱回话管理（Weak session management）
- 缓冲区溢出（Buffer overflow）
- 未恰当支持的HTTP方法（Improperly supported HTTP method）
- 远程命令执行（Remote command exection）
- 远程代码注入（Remote code injection）
- 带有漏洞的库（Vulnerable libraries）
- HTTP响应分割（HTTP response splitting）
- 跨站请求伪造（Cross Site Requset Forgery,CSRF）

3、模糊测试器类型

内存模糊测试（In memory fuzzing)

- 通过程序快照，实现数据的多次注入
- 执行完一个测试用例后，将新测试用例直接写入内存，并恢复上次的内存快照注入新的数据
- 同样适用于涉及客户端和服务器之间的大量交互的模糊网络应用程序

- 优点
  - 快速
  - 捷径

- 缺点
  - 假象
  - 复杂

4、程序插桩原理及实例

- 在软件动态测试中，程序插桩是一种基本的测试手段，有着广泛的应用。
- 程序插桩方法：借助往被测程序中插入操作，来实现测试目的的方法，即向源程序中添加一些语句，实现对程序语句的执行、变量的变化等情况进行检查。了解一个程序在某次运行中所有可执行语句被覆盖的情况，或是每个语句的实际执行次数。
- 最简单的插桩：在程序中插入打印语句printf(“……”)语句。

4、程序插桩原理及实例

- 例：求取两个整数X和Y的最大公约数程序如下：

```
int  gsd (int X, int Y)
  {    int Q=X;
       int R=Y;
       while(Q!=R)
         {   if(Q>R)
                 Q=Q-R;
             else R=R-Q;
         }
         return Q;
  }
```

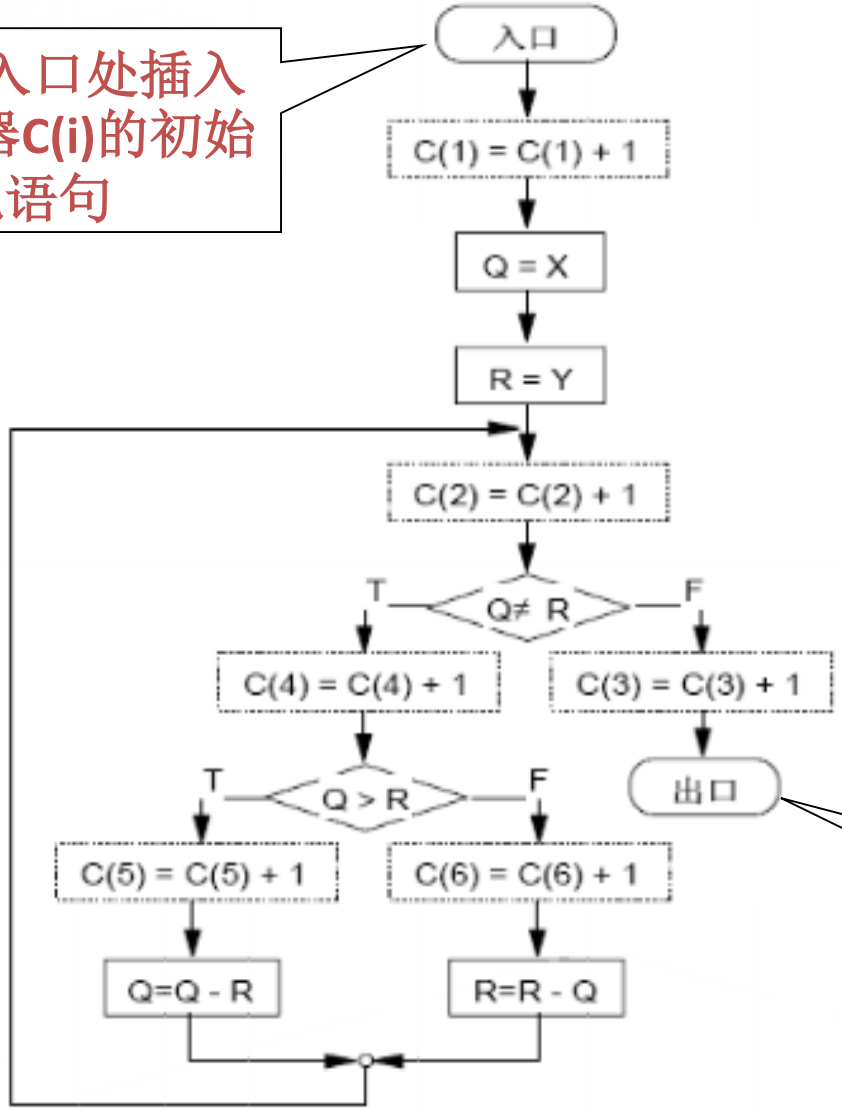
LISS
空天信息安全与可信计算
教育部重点实验室

4、程序插桩原理及实例

- 可以根据程序绘制出其流程图
- 为了记录改程序中语句的执行次数，我们使用插桩技术插入如下语句：
- C(i)=C(i)+1,i=1,2,…6
- 插桩之后的流程图如下：

4、程序插桩原理及实例

在程序入口处插入对计数器C(i)的初始化语句



在程序出口处加入打印语句

4、程序插桩原理及实例

- 图中虚线框中的内容并不是源程序的内容，而是我们为了记录语句的执行次数而插入的。虚线框中的程序就是为了完成计数。形式就是
- C（n）=C（n）+1 ； n = 1，2，3，……，n；
- 该程序从入口开始，到出口执行结束。凡经历的计数语句都能记录下该程序点的执行次数。
- 如果我们在程序的入口处还插入了对计数器C(i)初始化的语句，在出口处插入了打印这些计数器的语句，就构成了完整的插桩程序。它就能记录并输出在各程序点上语句的实际执行次数。

4、程序插桩原理及实例

Fuzzy程序插桩的方法：

1. 编译期间的插桩(需要源代码，例子：AFL)
2. 自定义调试器，使用断点来计算代码覆盖率(速度比较慢)。
3. 已经编译的程序上的动态插桩（无需源码，如PIN，Valgrind……）
4. 直接向二进制文件插入代码的静态插桩技术，使用的较少
5. 使用一些硬件特性：
   - IntelPT: 基于CPU硬件特性直接对二进制程序进行跟踪

4、程序插桩原理及实例

1、编译期间的插桩：以AFL为例

考虑如下的例子(x=argc):

```
if(x > 3) {
        puts("Test1\n");
} else {
        puts("Test2\n");
}
puts("Test3\n");
return 0;
```

```
user-VirtualBox# gcc -o test test.c
user-VirtualBox# ./test 1
Test2

Test3

user-VirtualBox# ./test 1 2 3 4 5 6
Test1

Test3
```

4、程序插桩原理及实例

编译期间的插桩：以AFL为例

考虑如下的例子(x=argc):

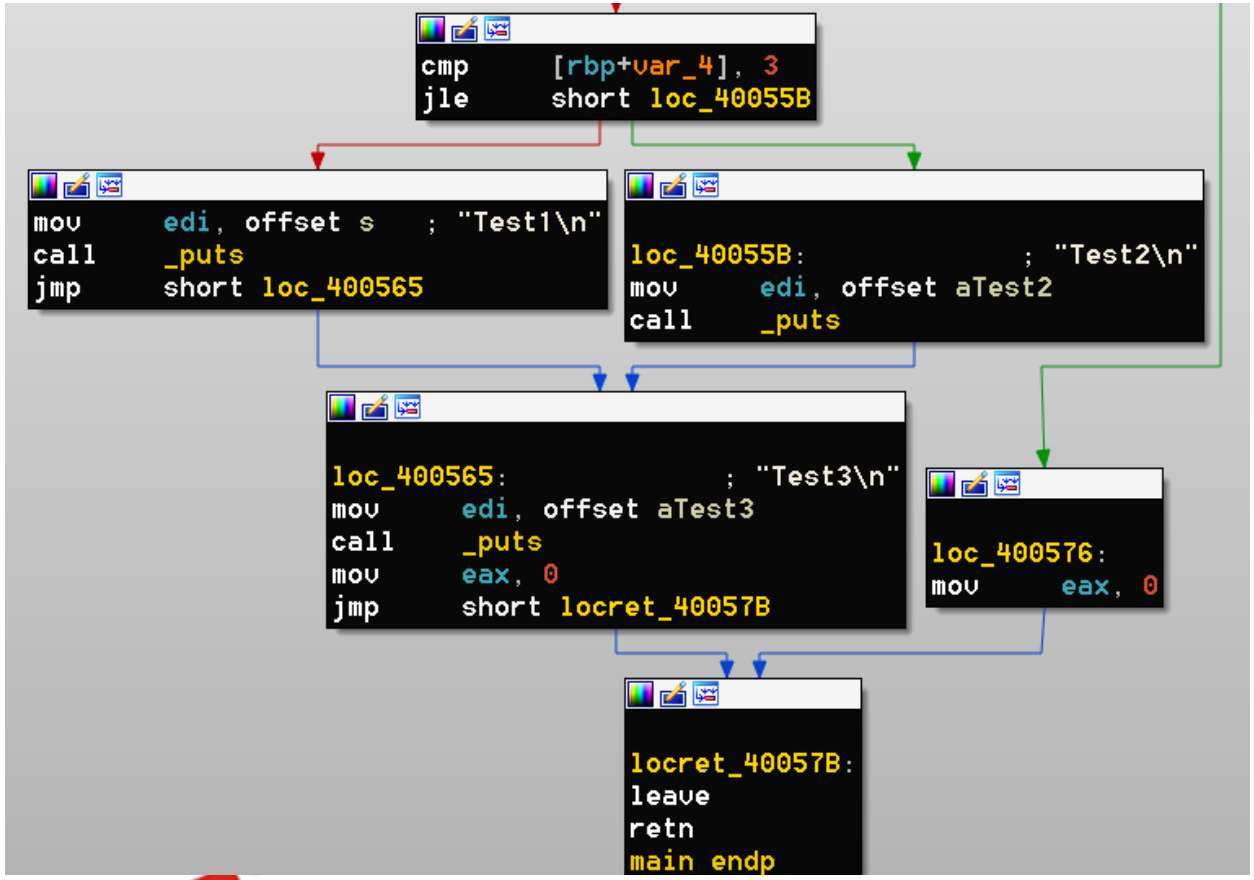
```
if(x > 3) {
        puts("Test1\n");
} else {
        puts("Test2\n");
}
puts("Test3\n");
return 0;
```

```
user-VirtualBox# gcc -o test test.c
user-VirtualBox# ./test 1
Test2

Test3

user-VirtualBox# ./test 1 2 3 4 5 6
Test1

Test3
```

4、程序插桩原理及实例

编译期间的插桩：以AFL为例

Ida中每一块都是一个basic block



4、程序插桩原理及实例

编译期间的插桩：以AFL为例

用afl-gcc来编译程序：

```
user-VirtualBox# afl-gcc -o test2 test.c
afl-cc 2.35b by <lcamtuf@google.com>
afl-as 2.35b by <lcamtuf@google.com>
[+] Instrumented 6 locations (64-bit, non-hardened mode, ratio 100%).
user-VirtualBox# ./test2 1
Test2

Test3

user-VirtualBox# ./test2 1 2 3 4 5
Test1

Test3
```

4、程序插桩原理及实例

编译期间的插桩：以AFL为例

保存寄存器的值

插桩

恢复寄存器的值

```
nop     dword ptr [rax]
lea     rsp, [rsp-98h]
mov     [rsp+0A0h+var_A0], rdx
mov     [rsp+0A0h+var_98], rcx
mov     [rsp+0A0h+var_90], rax
mov     rcx, 0BE80h
call    __afl_maybe_log
mov     rax, [rsp+0A0h+var_90]
mov     rcx, [rsp+0A0h+var_98]
mov     rdx, [rsp+0A0h+var_A0]
lea     rsp, [rsp+98h]
mov     edi, offset s    ; "Test2\n"
call    _puts
```

```
loc_4007E9:
argv = rsi                      ; char **
x = rdi                         ; int
nop     dword ptr [rax]
lea     rsp, [rsp-98h]
mov     [rsp+0A0h+var_A0], rdx
mov     [rsp+0A0h+var_98], rcx
mov     [rsp+0A0h+var_90], rax
mov     rcx, 55DDh
call    __afl_maybe_log
mov     rax, [rsp+0A0h+var_90]
mov     rcx, [rsp+0A0h+var_98]
mov     rdx, [rsp+0A0h+var_A0]
lea     rsp, [rsp+98h]
mov     edi, offset aTest1 ; "Test1\n"
call    _puts
jmp     loc_40079E
```

4、程序插桩原理及实例

编译期间的插桩：以AFL为例

插桩代码的主要作用。计算代码覆盖率：

```
cur_location = <compile_time_random_value>;
bitmap[(cur_location ^ prev_location) % BITMAP_SIZE]++;
prev_location = cur_location >> 1;
```

AFL会将_afl_prev_loc(之前的位置)同当前位置的key异或，之后当前的key右移一位，作为下一个的__afl_prev_loc，这个右移是一个很巧妙的设计，如果代码块的跳转为A->B或B->A，直接对两个Key异或结果是一样的，因此右移可以区分出一些特殊情况。接着bitmap表中对应的索引加一，表明一次hit。

4、程序插桩原理及实例

2. 使用断点

缺点：

- 速度非常慢
  - 原因之一是由于调试器的进程切换。
  - 只能粗略计算代码覆盖率，不能计算代码块的命中次数，不然会非常慢。

优点：

- 在逆向工程中可能会有作用，比如在每一个断点处dump寄存器的值。
- 适当利用可以优化测试效率
  - 可以只在未覆盖的代码块设置断点，在程序崩溃或者结束之前如果触发了断点，就可以知道本次运行覆盖了新的代码块。

LISS
空天信息安全与可信计算
教育部重点实验室

4、程序插桩原理及实例

3. 动态插桩

- 动态运行时操作正在运行的应用程序的指令
- 动态二进制插桩框架会直接在二进制文件或可执行文件中运行
- 不需要程序的源代码

4、程序插桩原理及实例

3. 动态插桩-----PIN

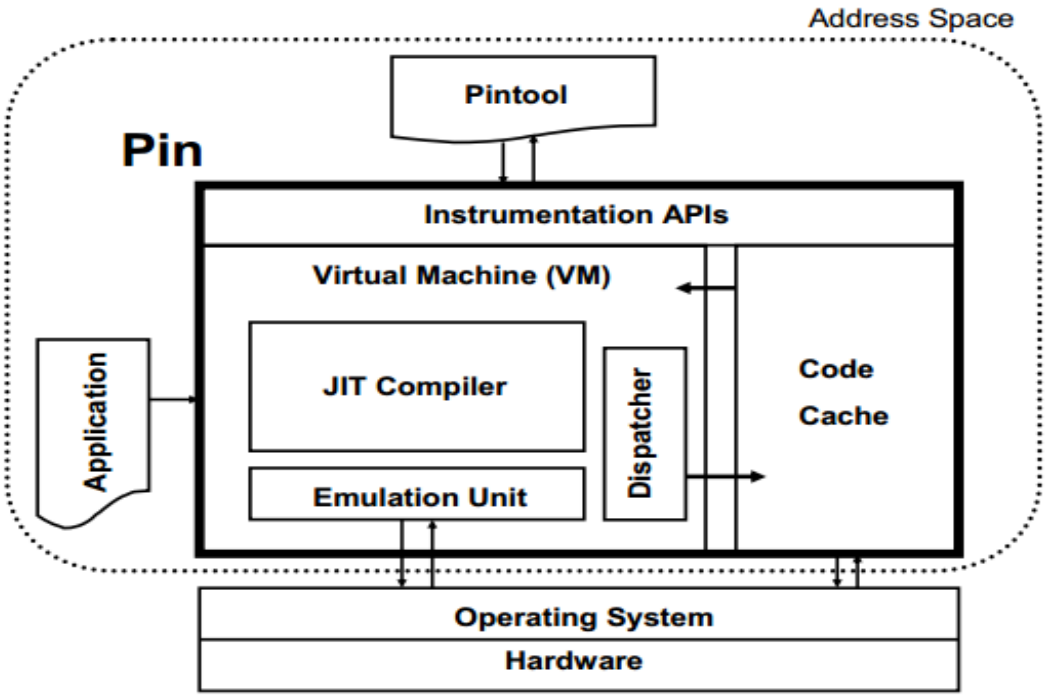
一种动态二进制检测框架，适用于x86,x64架构，一般用于程序动态分析，支持Windows 、Linux等平台。

- **PIN的主要功能：**
  - 替换原有程序函数
  - 探测程序任意指令，在自己设定的插入位置插入自己的代码并执行
  - 记录程序调用，包括syscall(检测改变参数)
  - 记录程序线程活动情况
  - 监测进程树
  - 模拟api调用

4、程序插桩原理及实例

3. 动态插桩-----PIN

- PIN包含了VM，code cache和instrumentation API.
  - 其中VM包含了一个just-intime compiler (JIT)，一个emulator和一个dispatcher.



4、程序插桩原理及实例

3. 动态插桩-----PIN

- VM是Pin的核心组成部分，负责调度插装代码的执行
- VM中包含：JIT Compiler、Emulation Unit、Dispatcher
  - ✓ JIT Compiler主要是对源程序指令进行即时重编译
  - ✓ Emulation Unit来解释一些不能被直接执行的指令
  - ✓ Dispatcher用来启动JIT编译以及检测程序代码
- Code Cache是存放即时编译所生成的新二进制代码的缓存空间
- Instrumentation API是由Pin提供给pintool，方便pin与pintool通信的接口

4、程序插桩原理及实例

4. 动态插桩-----Intel PT(Intel Processor Tracing)

- Intel研发的基于CPU硬件特性进行二进制程序跟踪的功能
- PT可以返回CPU中的数据流操作及地址信息
- 通过PT部件可以获得两类信息：基本执行信息（时间戳计时器、数据流边界等）和控制流信息（时间、程序执行流等）
- 支持LINUX
- 在无源码动态获取程序执行路径时，比QEMU模拟方式性能好

5、模糊测试工具及实例

AFL（American fuzzy lop）

是一个面向安全的fuzzer，它使用编译时插桩和遗传算法来自动发现测试用例，从而大大提高了模糊测试的代码覆盖率。

```
                american fuzzy lop 0.47b (readpng)

 process timing                                      overall results
        run time : 0 days, 0 hrs, 4 min, 43 sec      cycles done : 0
   last new path : 0 days, 0 hrs, 0 min, 26 sec      total paths : 195
 last uniq crash : none seen yet                    uniq crashes : 0
  last uniq hang : 0 days, 0 hrs, 1 min, 51 sec       uniq hangs : 1
 cycle progress                        map coverage
  now processing : 38 (19.49%)            map density : 1217 (7.43%)
 paths timed out : 0 (0.00%)           count coverage : 2.55 bits/tuple
 stage progress                        findings in depth
  now trying : interest 32/8           favored paths : 128 (65.64%)
 stage execs : 0/9990 (0.00%)           new edges on : 85 (43.59%)
 total execs : 654k                    total crashes : 0 (0 unique)
  exec speed : 2306/sec                  total hangs : 1 (1 unique)
 fuzzing strategy yields                             path geometry
   bit flips : 88/14.4k, 6/14.4k, 6/14.4k              levels : 3
  byte flips : 0/1804, 0/1786, 1/1750                 pending : 178
 arithmetics : 31/126k, 3/45.6k, 1/17.8k             pend fav : 114
  known ints : 1/15.8k, 4/65.8k, 6/78.2k             imported : 0
       havoc : 34/254k, 0/0                          variable : 0
        trim : 2876 B/931 (61.45% gain)                latent : 0
```

- http://lcamtuf.coredump.cx/afl/

5、模糊测试工具及实例

AFL（ American fuzzy lop）

① Process timing: Fuzzer运行时长、以及距离最近发现的路径、崩溃和挂起经过了多长时间。
② Overall results：Fuzzer当前状态的概述。
③ Cycle progress：输入测试用例队列处理的情况。
④ Map coverage：目标二进制文件中的插桩代码所观察到覆盖范围的细节。
⑤ Stage progress：关于突变策略产生的最新行为和结果的详细信息。执行次数和执行速度。
⑥ Findings in depth：有关找到的执行路径，异常和挂起数量的信息。
⑦ Fuzzing strategy yields： Fuzzer现在正在执行的文件变异策略。
⑧ Path geometry：有关Fuzzer找到的执行路径的信息。
⑨ CPU load：CPU利用率

- http://lcamtuf.coredump.cx/afl/

5、模糊测试工具及实例

Peach

由Deja vu Security公司的Michael Eddington创造并开发，是一个遵守MIT开源许可证的模糊测试框架，是第一款综合的开源fuzzing工具。它有两种主要模式，基于生成的模糊测试和基于变异的模糊测试。

```
[[ Peach v3.1.124.0
[[ Copyright (c) Michael Eddington
This is the Peach Runtime.  The Peach Runtime is one of the many ways
to use Peach XML files.  Currently this runtime is still in development
but already exposes several abilities to the end-user such as performing
simple fuzzer runs and performing parsing tests of Peach XML files.

Please submit any bugs to https://forums.peachfuzzer.com.

Syntax:

  peach -a channel
  peach -c peach_xml_file [test_name]
  peach [--skipto #] peach_xml_flie [test_name]
  peach -p 10,2 [--skipto #] peach_xml_file [test_name]
  peach --range 100,200 peach_xml_file [test_name]
  peach -t peach_xml_file

  -1                          Perform a single iteration
  -a,--agent                  Launch Peach Agent
  -c,--count                  Count test cases
  -t,--test xml_file          Validate a Peach XML file
  -p,--parallel M,N           Parallel fuzzing.  Total of M machines, this
                              is machine N.
```

https://sourceforge.net/projects/peachfuzz/

5、模糊测试工具及实例

Peach

优点：

- 模糊测试运行数百万自动化测试案例的创建、复制和登录漏洞。
- 根据常见的网络协议，使用智能测试用例模糊测试数据和数据流，测试范围复杂。
- 使用标准接口，可以支持使用任何格式、网络协议、应用协议，Android设备，装置驱动程序或嵌入式硬件的安全测试。
- 提供端到端的功能。可以支持本地和远程测试。
- 支持并行运算，能够在多台机器上进行并行处理，提高模糊测试的效率。

5、模糊测试工具及实例

Peach

Peach 的测试对象几乎包括了所有常见的Fuzz对象，例如文件结构，com，网络协议、API 等。

使用Peach进行fuzzing的主要步骤如下：

1、创建模型
2、选择/配置Publisher
3、配置代理/监视器
4、配置记录

5、模糊测试工具及实例

Peach的使用

Peach把用于数据定义的文件叫做Peach pit file。使用Peach时，实际上主要工作就是定义这样一个xml文件指示Peach测试平台去做测试。Peach pit file基本上总是包含以下几个部分：

```
<?xml...版本，编码之类...>
<Peach ...版本，作者介绍之类...>
<Include ...包含的外部文件/>
<DataModel >原始数据结构定义</DataModel>
<StateModel >测试逻辑，状态转换定义，如收到什么样的数据包之后，发出什么样对应的数据包</StateModel>
<Agent >检测 exception，crash 等 </Agent>
<Test >指定将要使用到的 state，agent，publisher 等</Test>
<Run >Fuzzer 执行的进入点</Run>
</Peach>
```

5、模糊测试工具及实例

Sanitizers

Sanitizers是谷歌发起的开源工具集，包括了AddressSanitizer, MemorySanitizer, ThreadSanitizer, LeakSanitizer等。

Sanitizers项目本是LLVM项目的一部分，但GNU也将该系列工具加入到了GCC编译器中。

GCC从4.8版本开始支持Address和Thread Sanitizer，4.9版本开始支持Leak Sanitizer和UB Sanitizer，这些都是查找隐藏Bug的利器。

5、模糊测试工具及实例

Sanitizers

Sanitizers主要用于内存错误的查找。内存访问错误是模糊测试通常能够发现的一类错误，其主要是由于程序对不该进行读写操作的内存进行了操作，从而导致了应用程序的崩溃。

实例：以test.c来作为一个例子：

```
1.#include <stdio.h>
2.int main()
3.{
4.    int a[2] = {1, 0};
5.    int b = a[2];
6.return 0;
7.}
```

5、模糊测试工具及实例

Radamsa

Radamsa是用于健壮性测试（又称为模糊测试）的测试用例生成器。它通常用于测试程序对格式错误和潜在恶意输入的承受能力。它通过读取有效数据的样本文件并从中生成各种不同输出来工作。

radamsa的主要特点是，它已经在真正重要的程序中发现了许多错误，它易于编写脚本，并且易于启动和运行。

5、模糊测试工具及实例

Radamsa使用

使用举例

Hello World!

echo ’Hello World!’ |

radamsa

```
root@iZbp1bsk7kfu18um8m510fZ:~/frankyu/radamsa-develop# echo 'Hello world!' | radamsa
Hello world!
Hello world!
Hello world!
root@iZbp1bsk7kfu18um8m510fZ:~/frankyu/radamsa-develop# echo 'Hello world!' | radamsa
Hello waaaa
```

5、模糊测试工具及实例

Honggfuzz介绍

- honggfuzz是由谷歌开发的，和AFL类似的反馈驱动，使用遗传算法来变异的fuzzer。
- honggfuzz是一个多进程和多线程的fuzzer，所以用honggfuzz进行fuzzing的速度非常快，在安全漏洞发现中的表现十分突出。

5、模糊测试工具及实例

Honggfuzz使用

1. 对程序进行源码插桩编译

export CC=/path/to/honggfuzz/hfuzz-cc/hfuzz-gcc
CXX=/path/to/honggfuzz/hfuzz-cc/hfuzz-g++
honggfuzz/hfuzz-cc目录下提供了编译器的wrapper，其实也可以设置CC， CXX为hfuzz-cc，因为其他的warapper都链接到hfuzz-cc

2. 用honggfuzz进行fuzz

假设已用hfuzz-cc插桩编译了软件xxxx, 并make install 到/usr/local/bin中。

./honggfuzz -f input_dir -P -z -- /usr/local/bin/xxxx-pv __FILE__

5、模糊测试工具及实例

Honggfuzz使用

```
------------------------[  0 days 03 hrs 36 mins 16 secs ]----------------------
  Iterations : 6,050,166 [6.05M]
        Mode : [2/2] Feedback Driven Mode
      Target : /usr/local/bin/exiv2 -pv __FILE__
     Threads : 1, CPUs: 1, CPU%: 100% [100%/CPU]
       Speed : 455/sec [avg: 466]
     Crashes : 0 [unique: 0, blacklist: 0, verified: 0]
    Timeouts : 0 [10 sec]
 Corpus Size : 1, max: 61,335 bytes, init: 100 files
  Cov Update : 0 days 03 hrs 36 mins 16 secs ago
    Coverage : edge: 0 pc: 0 cmp: 39
---------------------------------- [ LOGS ] -----------------/ honggfuzz 1.7 /-
[2018-11-23T00:45:21-0800][W][27587] arch_checkWait():308 Persistent mode: PID 77997 ex
 with status: EXITED, exit code: 255
Persistent mode: Launched new persistent PID: 77998
[2018-11-23T00:45:21-0800][W][27587] arch_checkWait():308 Persistent mode: PID 77998 ex
 with status: EXITED, exit code: 255
Persistent mode: Launched new persistent PID: 77999
[2018-11-23T00:45:21-0800][W][27587] arch_checkWait():308 Persistent mode: PID 77999 ex
 with status: EXITED, exit code: 255
```

5、模糊测试工具及实例

LibFuzzer

- LibFuzzer 是一个 in-process， coverage-guided ， evolutionary 的模糊测试引擎，它是 LLVM 项目的一部分。
- LibFuzzer 和要被测试的库链接在一起，通过一个特殊的模糊测试进入点（目标函数）从而将产生fuzz输入数据给到被测试的库。
- Fuzzer 会跟踪哪些代码区域已经测试过，然后在输入数据的语料库上产生变异，来最大化代码覆盖。代码覆盖的信息由 LLVM 的 SanitizerCoverage 插桩提供。

5、模糊测试工具及实例

LibFuzzer使用

如果要 Fuzz 一个程序，用户自己编写 LLVMFuzzerTestOneInput 函数。 Libfuzzer 生成的测试数据 Data以及测试数据的长度 Size，把这些生成的测试数据让程序来处理即可，LibFuzz 引擎同时会尽可能的触发更多的代码逻辑。 LibFuzzer 作为一个测试引擎负责生成这些测试数据，并且提供了一套异常检测机制。

```
// fuzz_target.cc extern "C" int LLVMFuzzerTestOneInput(const uint8_t
*Data, size_t Size) { DoSomethingInterestingWithMyAPI(Data, Size);
return 0; // Non-zero return values are reserved for future use. }
```

5、模糊测试工具及实例

LibFuzzer使用

•将待测对象和 libFuzzer.a 库编译链接

```
clang++ -g -std=c++11 -fsanitize=address -fsanitize-coverage=trace-pc-guard fourth_fuzzer.cc ../../libFuzzer/libFuzzer.a -o fourth_fuzzer
```

可以看到在编译时其中有很多前面提到的 Memory tool，Sanitizer 用于检测运行时出现的内存错误等异常。

•之后运行即可

```
mkdir corpus3 && ./fourth_fuzzer corpus3/ -max_len=1024
```

5、模糊测试工具及实例

Domato

- Domato 是由 Ivan Fratric 编写的 DOM fuzzer
- 从技术上讲，domato 不是一个真正的 fuzzer，而是一个测试用例生成器，因为它只能根据给定的，易于扩展的语法生成 HTML，CSS 和 JavaScript。

5、模糊测试工具及实例

Domato使用

生成 .html 样本：

```
python generator.py <output file>
```

通过单次调用生成多个样本：

```
python generator.py --output_dir <output directory> --no_of_files <number of output files>
```

生成的样本将被放置在指定的目录中，并将其命名为 fuzz-<number>.html，例如 fuzz-1.html，fuzz-2.html 等。生成多个样本则更快，因为输入语法文件只需加载和解析一次。

6、模糊测试的局限性

模糊测试的特点:

- Fuzzing测试的用例通常具备某种攻击性的畸形数据。
- 具备较好的自动化测试能力。
- “蛮力”攻击方法。
- 很少出现误报。
- 能够快速找到真正的漏洞，操作简单。
- 通常不是进行功能性测试，而是检查系统处理错误的能力，比如：“入侵”、“破坏”、“崩溃”
- 不能保证系统中已经没有漏洞。

6、模糊测试的局限性

模糊测试的缺点:

- 无法发现访问控制漏洞
- 逻辑设计不良
- 后门
- 无法发现多阶段安全漏洞
- 无法发现深入的漏洞（需要很多前置条件的）

实验　基于AFL的模糊测试

- 安装AFL
- 任意选择一个有源代码的样本
- 结合源代码分析用AFL进行模糊测试后的结果

问题？

谢　谢！

LISS
空天信息安全与可信计算
教育部重点实验室